

Towards Better Handling of Large Code Contexts: Improvements to Graph-based Generation

Akhmatov Bulat¹, Nekrasov Dmitrii², and Vabnits Alexandra³

¹b.akhmatov@innopolis.university

²d.nekrasov@innopolis.university

³a.vabnits@innopolis.university

Abstract

Modern language models struggle with large code repositories due to transformers' quadratic complexity and context selection challenges. We enhance GraphCoder by incorporating multiple embedding models (CodeT5, CodeBERT, GraphCodeBERT) and evaluating various distance metrics. Experiments with 38 configurations show our token-embedding hybrid improves upon GraphCoder, with CodeBERT excelling in code matching (0.73 EM) and GraphCodeBERT in identifier matching (0.87 F1). While Mamba integration failed due to lack of code-specific tuning, our results emphasize the need for specialized architectures.

1 Introduction

LLMs have transformed software development, but face two key challenges: (1) computational limits with long code contexts, and (2) relevant context identification in large repositories. Our contributions:

- Systematic evaluation of embedding-based retrieval (38 experiments in total)
- Improved GraphCoder with 1-3% gains via optimized retrieval
- Practical analysis of retrieval quality vs. computational overhead

2 Related Work

2.1 Transformers and Optimizations

Transformers (Vaswani et al., 2017) revolutionized NLP but suffer from $O(N^2)$ complexity. FlashAttention-2 (Dao et al., 2022) optimizes GPU operations. Mamba (Gu and Dao, 2023) offers linear complexity but requires code-specific tuning.

2.2 Graph-Based Approaches

RepoHyper(Wei Liu, 2024) introduces pattern search. Instead of full graph search, authors proposed to shrink scope only to include k nearest neighbours and then expand using exhausted strategy, collecting most frequent paths in graph. Then they rank candidates with link prediction model. GraphCoder(Luo, 2023) introduces graph builded from several types of dependencies: CFG - flow of execution based on conditional statement value (True/False), CDG - general flow of line executions, DDG - data flow in lines of code. Then it takes h -hops nodes with a limited amount of total taken nodes ($=k$). Ranking metric is jaccard index.

3 Methodology

Our methodology follows a multi-stage approach designed to evaluate, refine, and combine emerging architectures for long-context code completion in repository-level tasks. The primary baseline for our experiments is **GraphCoder**, due to its compatibility with modular enhancements.

We begin by setting up individual implementations of Mamba, GraphCoder, RepoHyper, Hierarchical Context Pruning. Both GraphCoder, RepoHyper, and HCP repositories appeared to be "broken". The first one was fixed and became the basis of our repository, <https://github.com/Brvcket/GraphCoder>; however, attempts to fix the other 2 were abandoned due to the similarity of ideas and the need to produce some results within the specified timeframe.

3.1 Embedding distance

Original GraphCoder uses the Jaccard coefficient on code tokens as an "embedding distance." We extend this idea by incorporating multiple distance and similarity metrics based on embeddings produced by three different embedder models:

1. Salesforce/codet5p-110m-embedding

2. microsoft/codebert-base
 3. microsoft/graphcodebert-base
 Let:

- $d_{t,i}$ = distance between target node and node i
- $w_{t,i}$ = weight between nodes
- τ = distance threshold
- $\mathbf{e}_t, \mathbf{e}_i$ = embedding vectors
- S_t, S_i = sets of code tokens
- $\text{sim}(\cdot, \cdot)$ = similarity function

The distance function is defined as:

$$d_{t,i} = w_{t,i} = \begin{cases} 0, & w_{t,i} < \tau \\ w_{t,i}, & w_{t,i} \geq \tau \end{cases}$$

The LLM completion:

$$f(w_{t,i} \times S) = \text{missing code}$$

We evaluated the following distance metrics for four types of token-embedding interaction:

Format:

Coarse-grained Method – Fine-grained Method

Coarse-grained Phase (Phase 1):

Purpose: Efficiently reduce search space from thousands to ~ 100 candidates simply ranking them by similarity

Fine-grained Phase (Phase 2):

Purpose: Re-rank top- k candidates ($k \ll n$) with context-aware precision using subgraph edit similarity and taking into attention interaction of 2 codes

1. **Token-Token:** Jaccard similarity applied to sets of tokens.
2. **Token-Embed:** Jaccard similarity on tokens combined with cosine distance (or other specified distances) applied to embeddings.
3. **Embed-Token:** Cosine distance (or other specified distance) applied to embeddings combined with Jaccard similarity on tokens.
4. **Embed-Embed:** Cosine distance (or other specified distances) applied to embeddings of both nodes.

For each of these interaction types, we utilized three different embedder models that were mentioned before.

Specific distance metrics evaluated:

- **Jaccard similarity** (on token sets):

$$J(S_t, S_i) = \frac{|S_t \cap S_i|}{|S_t \cup S_i|}$$

- **Cosine similarity** (on embeddings):

$$\text{sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$$

- **L1 distance:**

$$d_{L1}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_1$$

- **L2 distance:**

$$d_{L2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2$$

- **Dot product:**

$$\text{sim}_{\text{dot}}(\mathbf{u}, \mathbf{v}) = \langle \mathbf{u}, \mathbf{v} \rangle$$

Having 3 embedders, 4 distance metrics and 3 types of token-embedding interaction (without token-token that doesn't use embedder) we get 36 experiments + 2 experiments for token-token and the one without retrieval.

We run these 38 experiments in parallel using 4 virtual machines with 4xTesla A100 and 112 CPU cores in total, using multithreading for code search. GPU parallelization is simple - we partition the experiments on each machine, and each machine computes only its part of experiments.

We chose float16 versions of embedders because of the difference in time (approximately 1.5 times faster), although performance is slightly worse (Figure 1)

We used 100 test samples for each experiment, these samples are subsample of RepoEval-Updated api-level tasks(Luo, 2023) (removed code line includes at least one intra-repository defined API invocation), and it took us about 40 computational hours to complete.

3.2 Evaluation Metrics

Following the original GraphCoder setup, we evaluate model performance using two groups of metrics:

- **Code match:** We report Exact Match (EM) and Edit Similarity (ES), where

$$\text{ES} = 1 - \frac{\text{Lev}(y, \tilde{y})}{\max(|y|, |\tilde{y}|)},$$

and Lev denotes the Levenshtein distance.

- **Identifier match:** We extract identifiers (e.g., variable and API names) from both y and $\tilde{y}$, and compute EM and F1 scores to assess semantic consistency.

3.3 Distance function parametrization

The distance function can be parameterized as a neural network g_θ :

$$d_{t,i} = g_\theta(\mathbf{e}_t, \mathbf{e}_i)$$

where θ are learnable parameters and g_θ is typically an MLP that inputs embeddings and outputs distance. Another approach is to predict not distance, but weight between node and similarities. We have tried to integrate such learned metric, assuming that good embeddings will output good between nodes similarity metric. While we integrating our code into GraphCoder, we faced some problems in implementation. All those aspects limiting gradient flow, which restricts us of using DL based function for metrics. Namely, they are:

- String values are used in `top_k_context`, limiting good loss function choices
- Not appropriate for deep learning usage of similarity thresholding

So parametrization is possible, but needs a lot of changes in the code.

3.4 Mamba experiments

Mamba was chosen to compare it with a transformer in a large context and to evaluate if there was any difference with the pruned and whole context. Compared to transformer results, they perform poorly. Also, there is little decrease in performance when evaluated without context pruning. It may perform poorly because general mamba from HuggingFace `state-spaces/mamba-1.4b-hf` was trained on a general dataset and does not have code-specific fine-tuning. There is Mamba Codestral 7B tuned for coding tasks, but we failed to configure needed requirements on CUDA 11.8, which was installed on the virtual machine.

4 Results

4.1 Embedding distance

Method	Code Match		Identifier Match	
	EM	ES	EM	F1
W/o retrieval	0.39	0.68	0.39	0.66
GraphCoder	0.70	0.86	0.71	0.85
Ours	0.71	0.88	0.72	0.86

Table 1: Comparison of GraphCoder and token-embed on codet5p for code and identifier matching tasks. Best results are bolded.

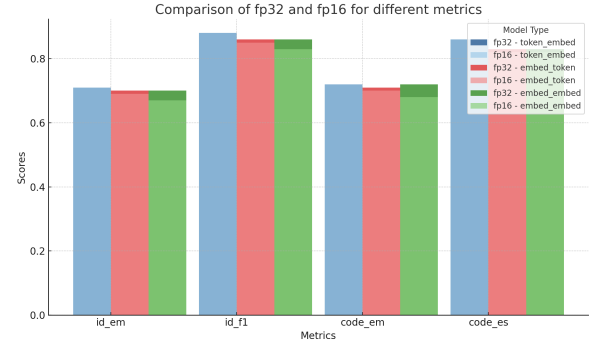


Figure 1: Comparison of float16 and float32 embedder (codet5p)

4.2 Mamba experiments

Method	Code Match		Identifier Match	
	EM	ES	EM	F1
W/o retrieval	0.02	0.28	0.03	0.25
GraphCoder	0.02	0.31	0.06	0.28

Table 2: Results for code and identifier matching tasks with mamba.

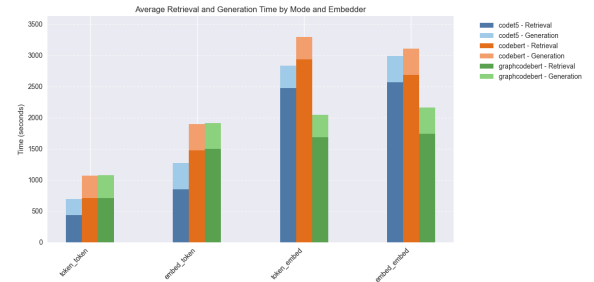


Figure 2: Retrieval and generation time with different embedders on 100 tasks

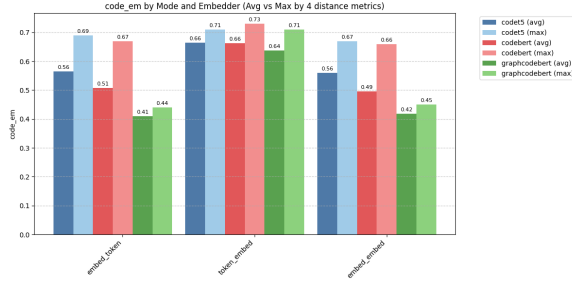


Figure 3: Code Exact Match (avg and max by distance metrics)

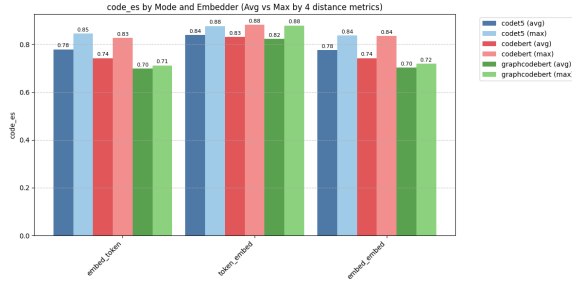


Figure 4: Code Edit Similarity (avg and max by distance metrics)

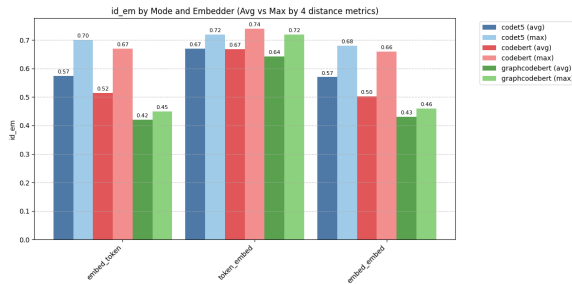


Figure 5: Identifier Exact Match (avg and max by distance metrics)

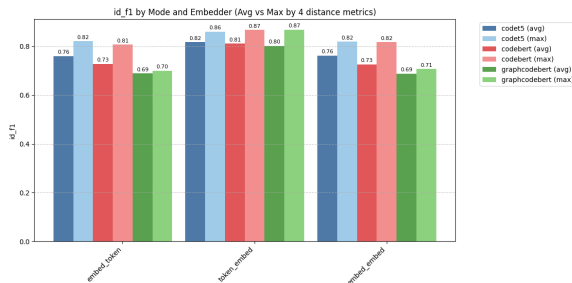


Figure 6: Identifier F1 (avg and max by distance metrics)

5 Discussion

Our experiments yield several key findings. First, while float32 embeddings show better performance

(Figure 1), we used float16 for practical efficiency. The timing results (Figure 2) confirm expected overhead from our enhanced retrieval methods, justified by improvements over GraphCoder (Table 1).

Analysis of embedding strategies (Figures 3-6) reveals:

- token_embed performs best overall
- CodeBERT excels in code matching (EM 0.73)
- GraphCodeBERT leads in identifier matching (F1 0.8681)
- Cosine similarity works best across metrics

The Mamba experiments failed to compete with transformers (Tables 1-2), likely due to lack of code-specific tuning. Attempts to learn distance metrics faced technical hurdles from the system’s discrete operations.

Our results suggest embedder choice should align with task priorities, with CodeBERT and GraphCodeBERT offering complementary strengths for different aspects of code completion. More details are in full experiment table in the appendix (Table 3)

References

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness.
- Albert Gu and Tri Dao. 2023. [Mamba: Linear-time sequence modeling with selective state spaces](#). *Preprint*, arXiv:2312.00752.
- Lei Zhang Yunshui Li Jiaming Li Xiaobo Xia Jiayi Yang Run Luo. 2023. [Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code llms](#). *Preprint*, arXiv:2406.18294.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). *Preprint*, arXiv:1706.03762.
- Daoguang Zan Bo Shen Wei Zhang Haiyan Zhao Zhi Jin Qianxiang Wang Wei Liu, Ailun Yu. 2024. [Graph-coder: Enhancing repository-level code completion via code context graph-based retrieval and language model](#). *Preprint*, arXiv:2406.07003.

A Appendix

Method	Code Match EM	Code Match ES	Identifier Match EM	Identifier Match F1
Without retrieval	0.3900	0.6813	0.3900	0.6649
Original retrieval	0.7000	0.8658	0.7100	0.8526
codet5				
cosine				
token_embed	0.7100	0.8771	0.7200	0.8607
embed_token	0.6900	0.8453	0.7000	0.8210
embed_embed	0.6700	0.8317	0.6800	0.8151
dot				
token_embed	0.7000	0.8632	0.7100	0.8435
embed_token	0.6900	0.8453	0.7000	0.8210
embed_embed	0.6700	0.8367	0.6800	0.8195
l2				
token_embed	0.6600	0.8320	0.6600	0.8115
embed_token	0.4400	0.7116	0.4500	0.7001
embed_embed	0.4500	0.7184	0.4600	0.7067
l1				
token_embed	0.5900	0.7826	0.5900	0.7553
embed_token	0.4400	0.7116	0.4500	0.7001
embed_embed	0.4500	0.7184	0.4600	0.7067
codebert				
cosine				
token_embed	0.7300	0.8822	0.7400	0.8679
embed_token	0.6700	0.8262	0.6700	0.8074
embed_embed	0.6600	0.8354	0.6600	0.8174
dot				
token_embed	0.6900	0.8560	0.7000	0.8417
embed_token	0.4400	0.6996	0.4500	0.6871
embed_embed	0.4200	0.6963	0.4300	0.6752
l2				
token_embed	0.6500	0.8115	0.6500	0.7928
embed_token	0.4600	0.7206	0.4700	0.7085
embed_embed	0.4500	0.7184	0.4600	0.7067
l1				
token_embed	0.5800	0.7743	0.5800	0.7453
embed_token	0.4600	0.7206	0.4700	0.7085
embed_embed	0.4500	0.7184	0.4600	0.7067
graphcodebert				
cosine				
token_embed	0.7100	0.8790	0.7200	0.8681
embed_token	0.3800	0.6853	0.3900	0.6767
embed_embed	0.3900	0.6918	0.4100	0.6764
dot				
token_embed	0.6800	0.8532	0.6900	0.8380
embed_token	0.3800	0.6860	0.3900	0.6790
embed_embed	0.3800	0.6805	0.3900	0.6643
l2				
token_embed	0.6200	0.8071	0.6200	0.7800
embed_token	0.4400	0.7116	0.4500	0.7001
embed_embed	0.4500	0.7184	0.4600	0.7067
l1				
token_embed	0.5400	0.7526	0.5400	0.7253
embed_token	0.4400	0.7105	0.4500	0.7001
embed_embed	0.4500	0.7184	0.4600	0.7067

Table 3: Performance comparison of different methods across various metrics on 100 tasks.